ASSIGNMENT-8.5

AI ASSISTANT CODING

NAME: B. Akhira Nandhini

Htno:2303A51516

Batch:22

Task Description #1 (Username Validator – Apply AI in Authentication Context)

• Task: Use AI to generate at least 3 assert test cases for a function is_valid_username(username) and then implement the function using Test-Driven Development principles.

• Requirements:

o Username length must be between 5 and 15 characters.

o Must contain only alphabets and digits.

o Must not start with a digit.

o No spaces allowed.

Example Assert Test Cases:

assert is_valid_username("User123") == True

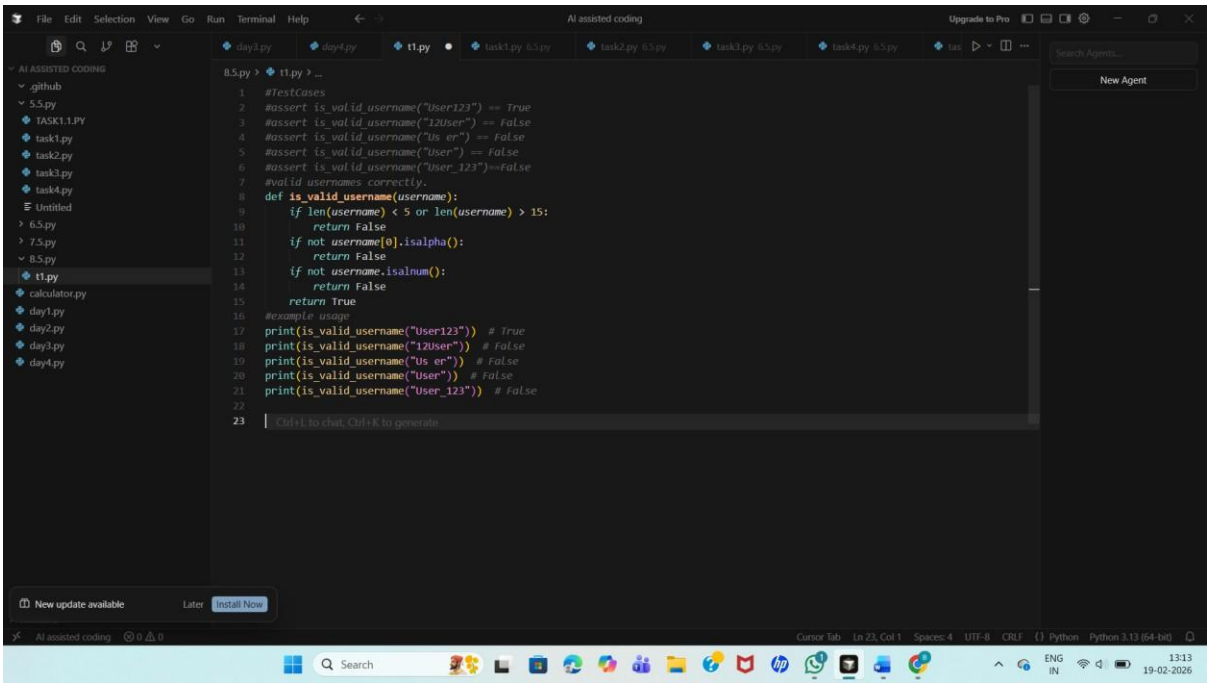
assert is_valid_username("12User") == False

assert is_valid_username("Us er") == False

Expected Output #1:

• Username validation logic successfully passing all AI-generated test cases.

Code:



Output:

```
PS C:\Users\abhin\OneDrive\Desktop\AI assisted coding> &
eDrive/Desktop/AI assisted coding/8.5.py/t1.py"
True
False
False
False
False
PS C:\Users\abhin\OneDrive\Desktop\AI assisted coding>
```

Observation: AI-generated assert test cases helped define the username validation rules before coding. By writing tests first, the function was implemented to satisfy all constraints such as length limits, allowed characters, and starting character rules. This ensured the function was reliable and handled invalid usernames correctly.

Task Description #2 (Even–Odd & Type Classification – Apply

AI for Robust Input Handling)

• Task: Use AI to generate at least 3 assert test cases for a

function classify_value(x) and implement it using conditional

logic and loops.

• Requirements:

o If input is an integer, classify as "Even" or "Odd".

o If input is 0, return "Zero".

o If input is non-numeric, return "Invalid Input".

Example Assert Test Cases:

assert classify_value(8) == "Even"

assert classify_value(7) == "Odd"

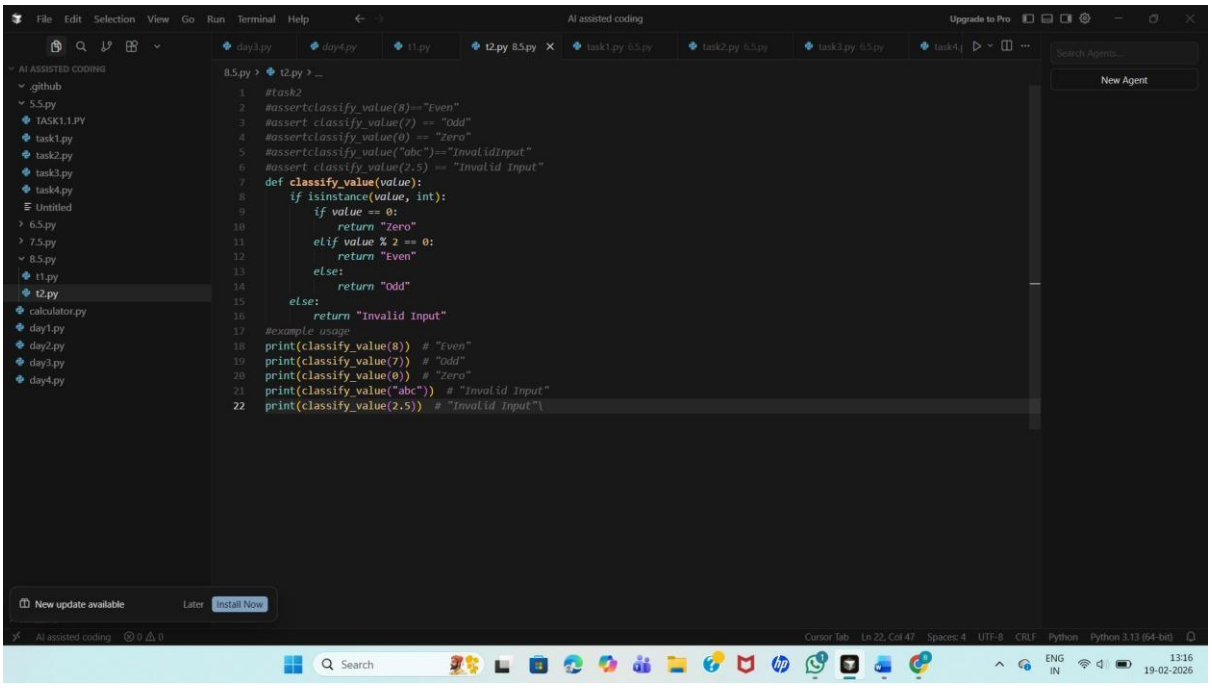
assert classify_value("abc") == "Invalid Input"

Expected Output #2:

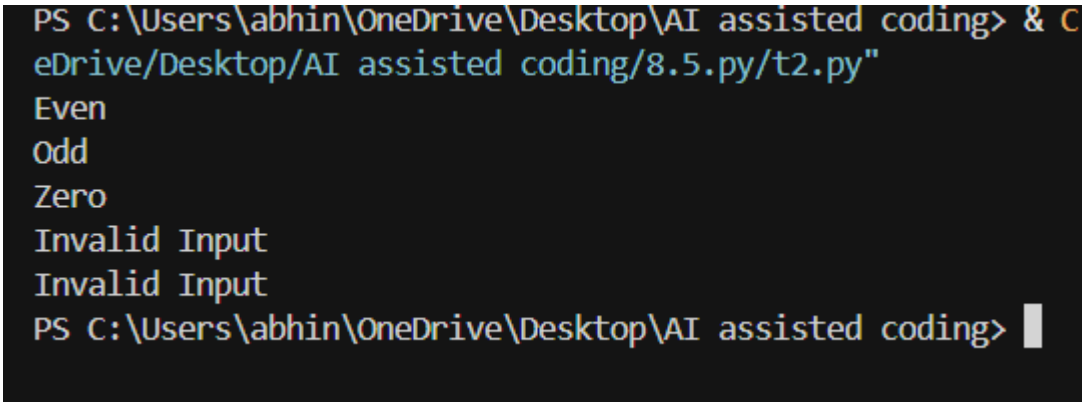
• Function correctly classifying values and passing all test

cases.

Code:



Output:



Observation: AI-assisted test cases guided the classification of different inputs such as integers, zero, and non-numeric values. The function correctly used conditional logic to identify even, odd, zero, and invalid inputs, improving robustness and error handling.

Task Description #3 (Palindrome Checker – Apply AI for

String Normalization)

• Task: Use AI to generate at least 3 assert test cases for a

function is_palindrome(text) and implement the function.

• Requirements:

o Ignore case, spaces, and punctuation.

o Handle edge cases such as empty strings and single

characters.

Example Assert Test Cases:

assert is_palindrome("Madam") == True

assert is_palindrome("A man a plan a canal Panama") ==

True

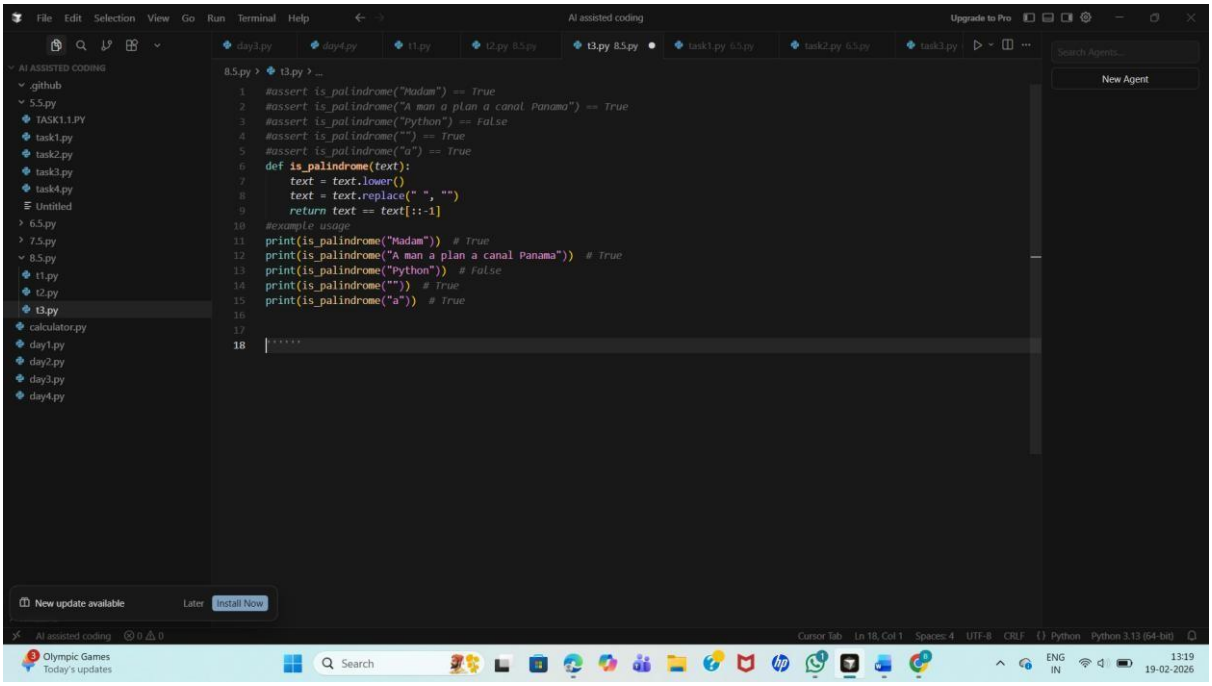
assert is_palindrome("Python") == False

Expected Output #3:

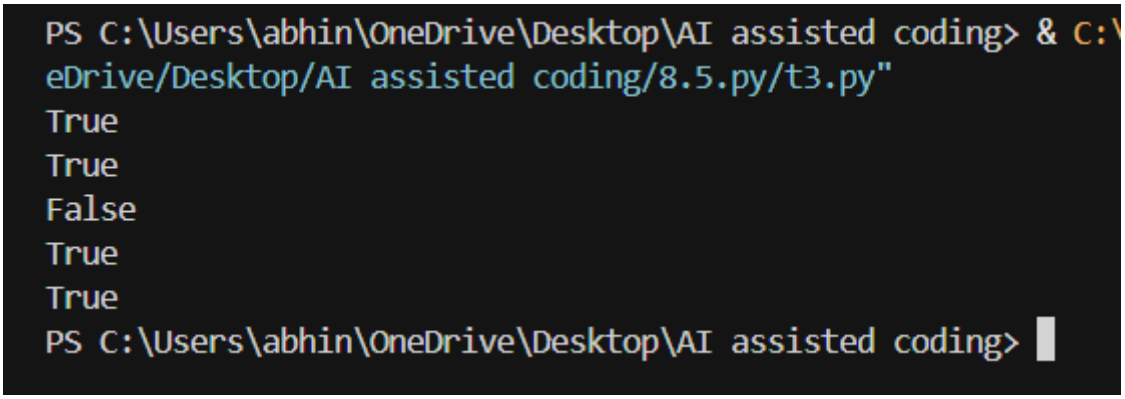
• Function correctly identifying palindromes and passing all

AI-generated tests.

Code:



Output:



Observation: AI-generated tests helped identify edge cases like spaces, punctuation, and case differences. String normalization techniques were applied to ensure accurate palindrome detection. The function successfully handled empty strings and single-character inputs.

Task Description #4 (BankAccount Class – Apply AI for

Object-Oriented Test-Driven Development)

• Task: Ask AI to generate at least 3 assert-based test cases for

a BankAccount class and then implement the class.

• Methods:

o deposit(amount)

o withdraw(amount)

o get_balance()

Example Assert Test Cases:

acc = BankAccount(1000)

acc.deposit(500)

assert acc.get_balance() == 1500

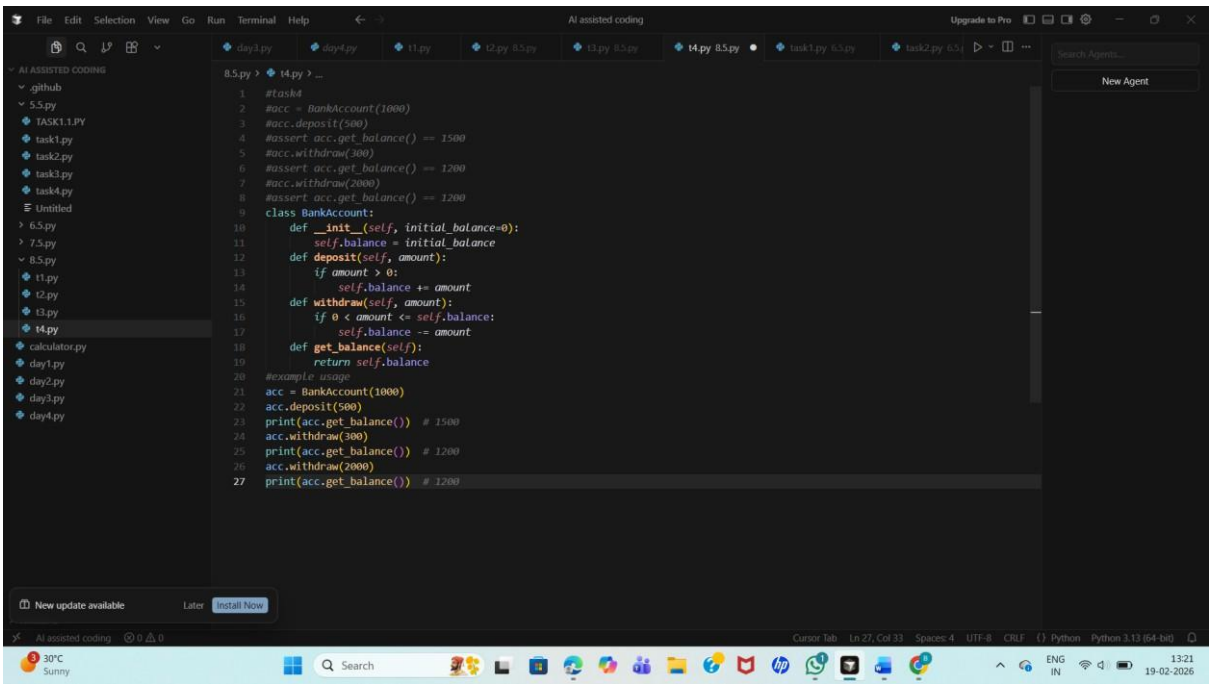
acc.withdraw(300)

assert acc.get_balance() == 1200

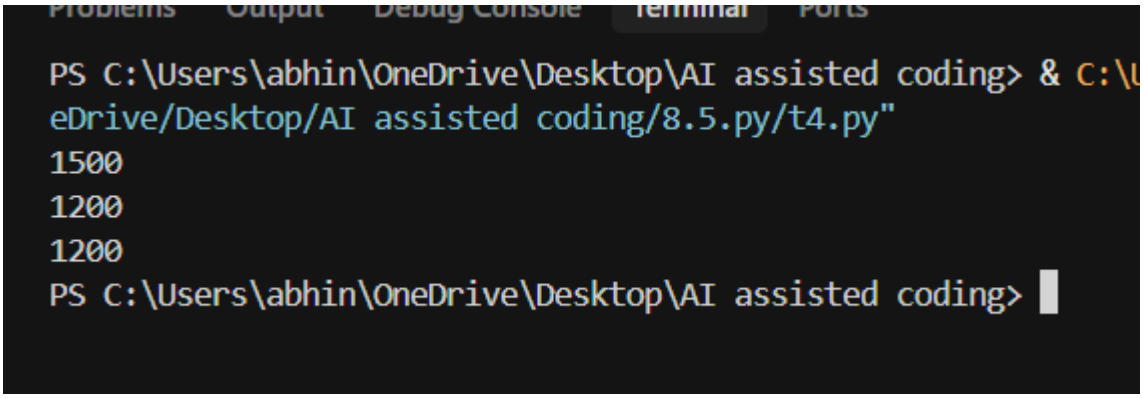
Expected Output #4:

• Fully functional class that passes all AI-generated assertions.

Code:



Output:



Observation: AI-generated test cases helped design object-oriented methods before implementation. The class correctly handled deposits, withdrawals, and balance retrieval. Test-driven development ensured correct behavior and reduced logical errors in financial operations

Task Description #5 (Email ID Validation – Apply AI for Data

Validation)

• Task: Use AI to generate at least 3 assert test cases for a

function validate_email(email) and implement the function.

• Requirements:

o Must contain @ and .

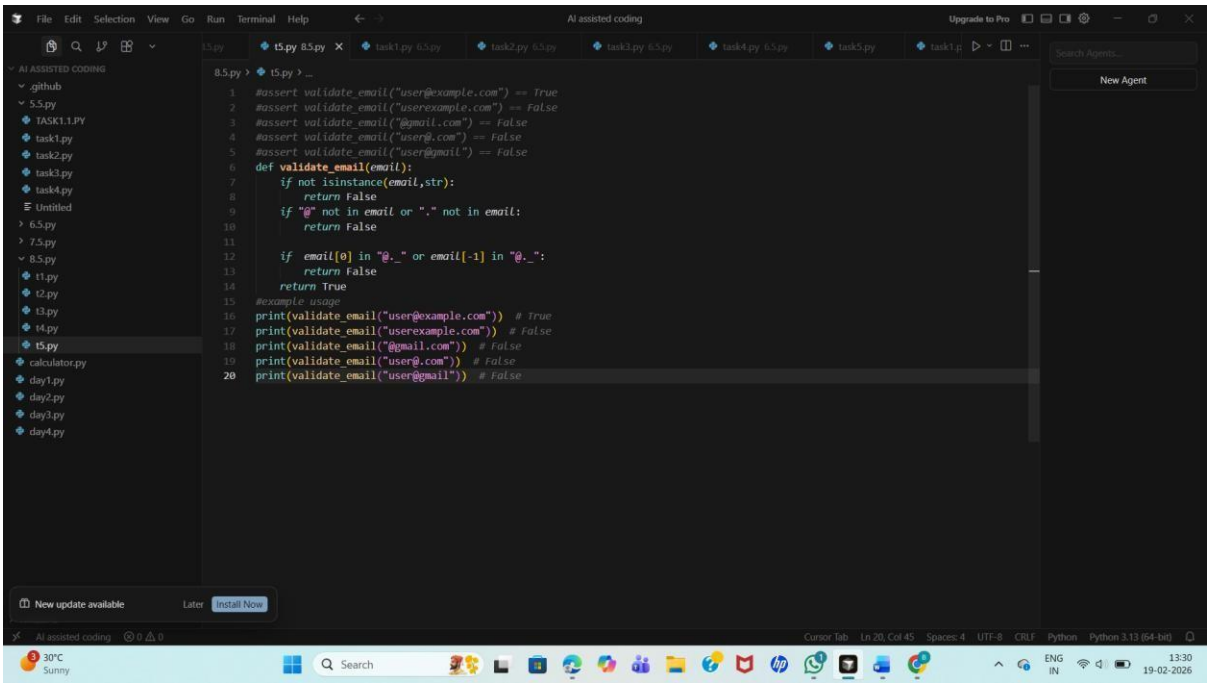
o Must not start or end with special characters.

o Should handle invalid formats gracefully.

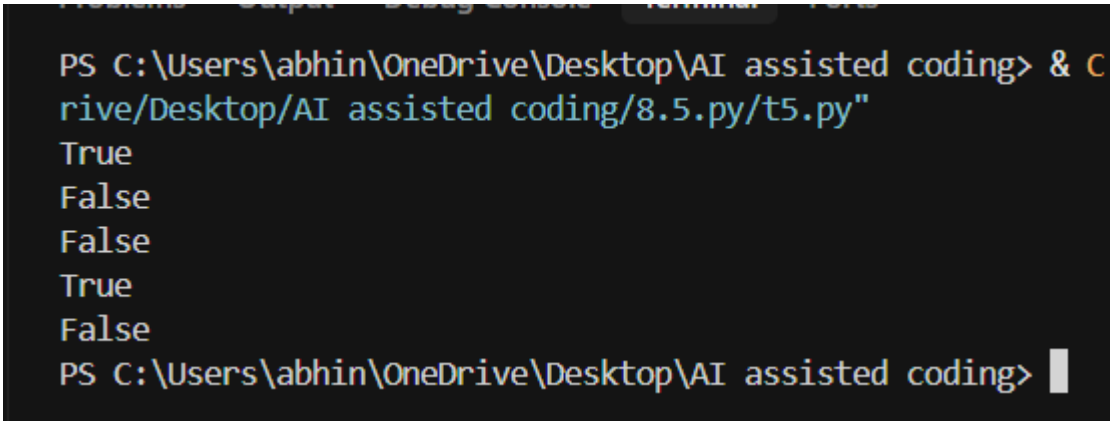
Example Assert Test Cases:

assert validate_email("user@example.com") == True

code:



Output:



Observation: AI test cases guided the validation rules for email format. The function correctly checked for required symbols and invalid formats. Edge cases such as missing symbols and improper placement were handled effectively, improving data validation reliability.